

Communications Systems Visualizer

ENGR 78 Final Project

Howard Wang

Kaw Moo

Charlie Schuetz

Spring 2026

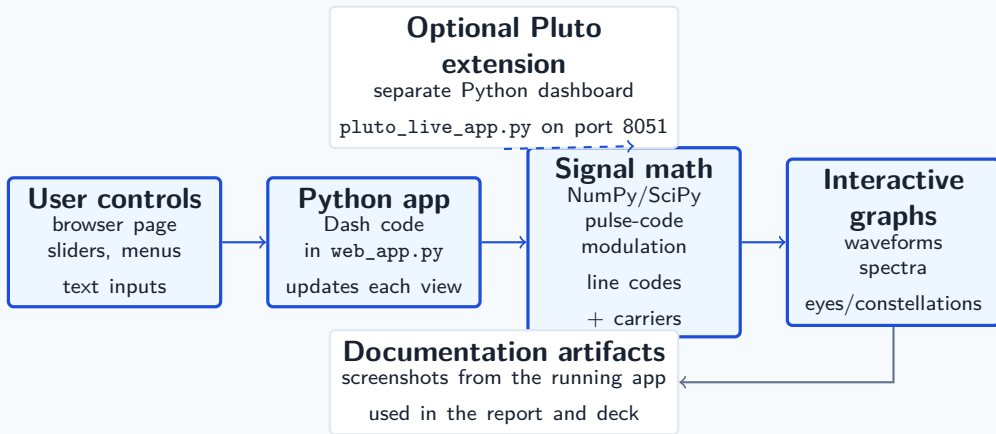
Motivation

- ENGR 78 spans sampling, baseband signaling, pulse design, passband modulation, and information limits—often taught as **separate** problem sets or static plots.
- Students benefit from **one place** where sliders and inputs change **linked** time-domain, frequency-domain, and statistical views at once.
- A locally hosted **Dash** app keeps the demo reproducible in any classroom or dorm room without bench hardware.

Project objective

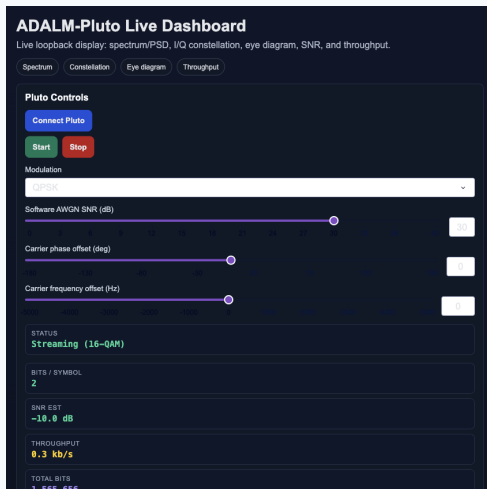
- 1 Build a **browser-based ENGR 78 learning visualizer**, not a hardware-dependent SDR lab.
- 2 Let users adjust course parameters—bits, bandwidth, roll-off, noise, symbol rate, and modulation order—and see **linked** waveforms, spectra, eye/constellation views, throughput, and capacity results.
- 3 Keep the core demo **laptop-only** with `python main.py`; treat **ADALM-PLUTO** live mode as a separate optional extension.

Overview: how the app works



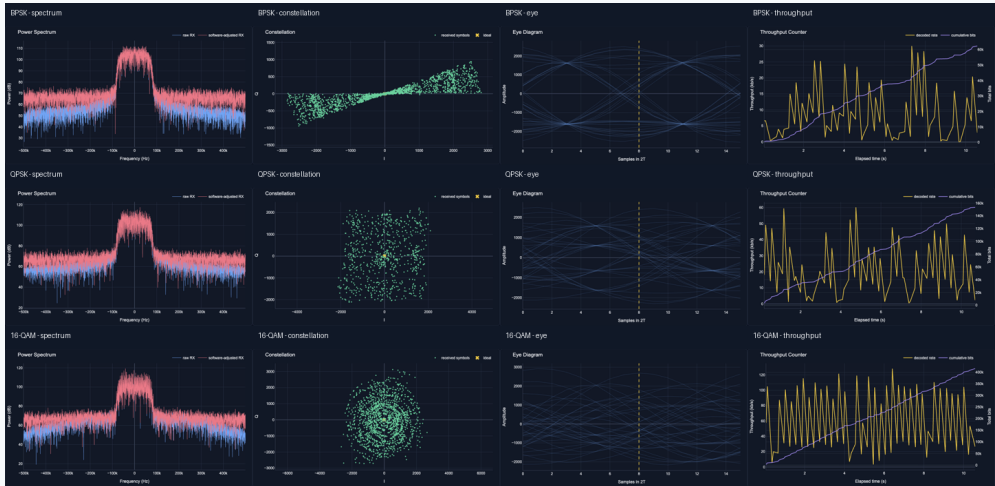
Core path: user changes a control → Python recomputes the signal data → Plotly redraws the graphs.

Dashboard (carrier tab: QPSK)



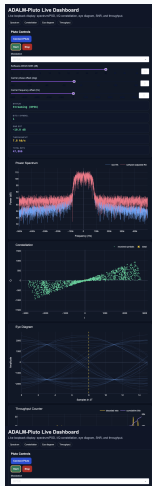
Tab navigation, summary statistics, and synchronized Plotly panels.

Composite plot export



Single image combining constellation, spectrum, eye, and throughput views for documentation.

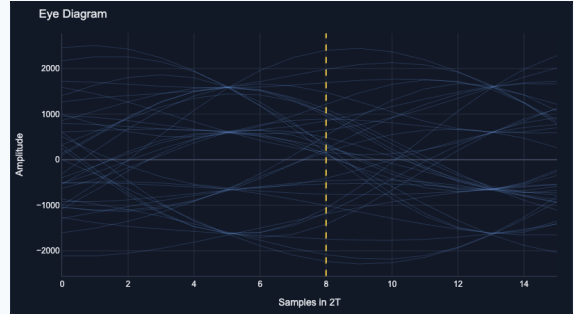
Full-dashboard capture (BPSK)



Constellation and eye diagram (QPSK)



Constellation: symbol map



Eye diagram

Tabs I—III: baseband and channel

- **PCM** — bandwidth B_m , oversampling vs. Nyquist, bits per sample n , implied PCM bit rate and waveform sketch.
- **Line coding** — user bit string (up to 32 bits), NRZ/RZ on-off and polar, AMI, Manchester; waveform + windowed PSD.
- **ISI + eye** — raised-cosine roll-off β , neighbor-symbol coupling, Gaussian noise, PAM order; overlaid eye segments after matched filtering.

Tabs IV—V: passband and information

- ***M*-ary + carrier** — PAM spacing, symbol rate R_s , ASK/OOK, BPSK, BFSK, QPSK, 16-QAM time/spectrum views; throughput and cumulative bits.
- **Capacity + codes** — Shannon $C = B \log_2(1 + \text{SNR})$ vs. SNR with operating point; discrete source entropy; repetition-code Hamming-distance toy.

Implementation: Core logic and callbacks live in `web_app.py`; `main.py` only starts the server.

Optional hardware path

- `pluto_live_app.py`: second Dash app (default **port 8051**) streaming IQ through `RealtimeEngine (src/realtime_engine.py)`. Use **Simulation (no radio)** without hardware, then **Start**.
- Typical schemes in the live UI: BPSK, QPSK, 16-QAM; spectrum, constellation, eye, SNR/throughput trends.
- macOS: Pluto USB Ethernet in `ncm` mode; device often at `192.168.2.1` (see repository `README.md`).

Course grading and demos can rely entirely on the browser app; Pluto is additive.

Key files

File	Role
<code>main.py</code>	Entry point; runs Dash on localhost (port 8050).
<code>web_app.py</code>	Tabs, callbacks, NumPy/Plotly signal generation
<code>pluto_live_app.py</code>	Optional Pluto live dashboard
<code>requirements.txt</code>	NumPy, SciPy, Dash, Plotly, pyadi-iio

Outcomes and extensions

- Delivered a **unified** interactive visualizer aligned with ENGR 78 topic progression.
- Students can explore rate, bandwidth, noise, and constellation geometry without installing a heavy SDR toolchain for the core lab.
- Natural extensions: saved presets per lecture, CSV export, additional codes or decoders, deeper FEC labs.

Credits and codebase size

Project team

Howard Wang
Kaw Moo
Charlie Schuetz

AI assistance

GPT 5.5 and Cursor Composer helped with code drafting, debugging, documentation, and presentation refinement.

Project scale

~4,200 lines
application source code

Count excludes virtual environments and generated files. It includes the Python app, `src/` modules, CSS, and source helper scripts.

Thank you

Questions?

Communications Systems Visualizer ■ ENGR 78 Final Project